智汇研途

AI 科研辅助平台 • 技术文档

2026 第九届传智杯全国大学生数智创新与 AI 应用大赛

AI 大模型创新应用挑战赛（个人赛）

参赛选手：_____

所在学校：_____

指导教师：_____

2026 年 11 月

一、作品概述

1.1 项目简介

「智汇研途」是一个基于多智能体协作与 RAG（检索增强生成，Retrieval-Augmented Generation）技术的科研辅助平台。针对通用大模型在学术场景中“幻觉频发、结论缺乏来源”的核心痛点，平台采用「检索员 → 总结员 → 批判员」三个智能体分工协作的工作流，先检索真实文献、再归纳总结、最后进行批判性校验，结合 ChromaDB 向量库实现语义检索，让每一次回答都尽可能“有据可查”。

1.2 设计目标

- 降低大模型幻觉：通过 RAG 检索与源头溯源机制，让回答基于真实文献而非凭空生成。
- 提升调研效率：用多智能体流水线替代人工「查文献—读文献—写综述」的重复劳动。
- 过程透明可追溯：将智能体的运行过程与引用来源可视化，工程透明度高。

1.3 核心亮点

- 多智能体协作：检索员、总结员、批判员三层质检，分工明确。
- 抗幻觉 RAG：ChromaDB 向量库 + 语义检索，缓解大模型幻觉。
- 工程可视化：智能体工作流实时动画 + 引用来源分布图表。
- 真实大模型接入：DeepSeek API，含超时重试与失败自动降级。
- 工程规范：前后端分离、.env 密钥管理、细粒度 Git 提交历史、完整文档。

二、需求分析

2.1 科研场景痛点

- 信息过载：论文数量爆发式增长（如 arXiv 日均更新数百篇），人工检索与筛选效率低下。
- 大模型幻觉：通用 Chatbot 会“一本正经地编造文献”，无法满足学术场景对确定性的要求。
- 结论不可核验：普通 AI 回答缺少出处与位置锚点，难以判断真伪。

2.2 目标用户

高校师生、科研工作者等需要频繁进行文献调研、综述撰写与学术思路梳理的人群。

三、系统架构设计

3.1 总体架构

系统采用前后端分离架构，自上而下分为四层：

- 表现层（前端）：原生 HTML + TailwindCSS + Chart.js，负责交互界面、消息渲染与工作流可视化。
- 服务层（后端）：FastAPI 提供 RESTful 接口，封装 DeepSeek API 调用与 RAG 检索。
- 检索层（向量库）：ChromaDB 持久化存储文献片段，提供语义相似度检索。
- 模型层：DeepSeek Chat API 提供大模型推理能力。

前端通过 HTTP 调用后端 /api/chat 等接口；后端负责鉴权、错误重试与检索增强，前后端解耦、职责清晰。

3.2 多智能体协作架构

平台将一次科研问答拆解为三个智能体的接力协作：

- 检索员（Retriever）：负责从文献库中检索与问题相关的文献片段，输出候选证据。
- 总结员（Summarizer）：基于检索到的证据进行归纳整理，形成结构化回答。
- 批判员（Critic）：对回答进行逻辑一致性检查与引用真实性校验，剔除不可靠结论。

三个智能体依次执行，上一环节的输出作为下一环节的输入，形成“检索—总结—批判”的质量闭环。

3.3 RAG 检索流程

文档切片 → 向量化 → 相似度检索 → 增强生成。系统将文献切分为片段并向量化存入 ChromaDB，收到用户问题后先做语义检索召回相关片段，再将其作为上下文交给大模型生成回答，从而降低幻觉。

四、技术选型

层级	技术	说明
前端	HTML + TailwindCSS	响应式布局与现代化 UI
前端	Font Awesome	图标库
前端	Chart.js	引用来源分布甜甜圈图
后端	FastAPI	高性能 Python Web 框架

后端	Pydantic	请求/响应数据校验
向量库	ChromaDB	文献片段向量化与语义检索
大模型	DeepSeek Chat API	真实推理能力接入
辅助	python-dotenv	环境变量与密钥管理

五、核心功能实现

5.1 前端界面与响应式布局

包含顶部 Header（Logo、系统标题、模型状态指示点）、左侧边栏（新建课题、项目历史、版权信息）、主工作区（课题标题栏、标签、导出按钮、聊天区、输入区）以及右侧智能体工作流面板。通过 TailwindCSS 实现响应式布局，小屏自动隐藏侧边栏与面板。

5.2 聊天交互与消息渲染

实现全局状态管理（messages / isProcessing / currentStep）、用户与 AI 消息气泡渲染、Markdown 基础渲染、打字加载动画、Enter 发送与 Shift+Enter 换行、新建课题切换标题、设置弹窗开关等交互逻辑。

5.3 多智能体工作流可视化

右侧面板以三张步骤卡片展示检索员、总结员、批判员的运行状态，通过 active / completed 状态动画实时呈现协作过程，把“看不见的推理过程”转化为“看得见的协作流程”，提升工程透明度。

5.4 引用来源分布可视化

使用 Chart.js 绘制引用来源分布甜甜圈图，直观展示 arXiv、PubMed、IEEE Xplore 等来源占比。

5.5 真实大模型接入

后端封装 DeepSeek Chat API 调用，支持超时控制（30 秒）与指数退避重试（最多 2 次）。前端优先调用真实接口，后端未启动时自动降级到模拟回复，保证演示稳定。

5.6 RAG 文献检索

后端集成 ChromaDB 向量库，提供 /api/rag/ingest（写入文献片段）与 /api/rag/search（语义检索）两个接口，采用确定性哈希嵌入实现离线可用的向量检索演示。

六、能力现状与迭代路线图

结合赛题六大命题模块，本项目当前版本聚焦「抗幻觉知识问答」这一核心场景，已完成多智能体协作、RAG 检索、真实大模型接入与工作流可视化；其余模块作为清晰的迭代路线，具备可扩展的架构基础。

已完成能力：

- 多智能体协作问答（检索员 / 总结员 / 批判员）
- RAG 文献检索（ChromaDB 向量库 + 语义检索接口）
- 真实 DeepSeek 大模型接入（超时重试 + 失败降级）
- 智能体工作流与引用来源可视化

迭代路线图：

赛题模块	当前状态	下一步计划
模块一：PDF 文件管理与细粒度解析	未覆盖	接入 PDF 解析 + 语义去重
模块二：抗幻觉知识问答	已实现核心	增强多文对比与溯源锚点
模块三：版式感知阅读与双向翻译	未覆盖	原生 PDF 渲染 + 中英联动
模块四：论文自动检索与推荐	部分（检索接口就绪）	对接 arXiv/PubMed API
模块五：引用图谱与综述生成	部分（来源可视化）	引用网络图 + 图论挖掘
模块六：写作辅助与可视化 Copilot	部分（结构化输出）	CSV 图表 + LaTeX 生成

评分维度对齐：

评分维度（权重）	本作品对应设计
Agent 与大模型技术深度（35%）	多智能体编排、RAG 抗幻觉、真实 API 接入与降级
科研场景创新性（25%）	三层质检 + 引用溯源可视化
全栈工程质量（10%）	前后端分离、错误重试、密钥安全、细粒度提交
科研实用性与闭环（10%）	真实问答闭环 + 边界测试
学术级 UI/UX（10%）	三栏布局、工作流动画、图表可视化
学术价值与广度（10%）	AI4S 范式、架构可扩展

七、关键技术

7.1 抗幻觉 RAG 与源头溯源

通过“先检索、后生成”的 RAG 策略，将真实文献片段作为生成上下文，从源头抑制幻觉；引用来源可视化使结论具备可追溯性。

7.2 多智能体任务编排

将复杂科研任务解耦为检索、总结、批判三个子任务，按序编排，降低单模型一次性生成出错的风险。

7.3 错误处理与优雅降级

后端对 DeepSeek API 的限流（429）与 5xx 错误实现指数退避重试；前端在后端不可用时自动降级到模拟回复，保证演示与使用体验稳定。

7.4 API 密钥安全管理

API Key 通过 .env 文件以环境变量方式注入，.env 已加入 .gitignore，确保密钥不出现在公开仓库。

八、测试与验证

8.1 功能测试用例

编号	测试项	预期结果	结果
1	空输入	弹出提示“请输入你的问题！”	通过
2	正常发送	显示用户气泡与 AI 回复	通过
3	Shift+Enter 换行	换行不发送	通过
4	连续发送	处理中按钮置灰，不重复发送	通过
5	新建课题	标题切换、聊天区清空	通过
6	设置弹窗	弹窗正常开关	通过
7	真实 API 调用	返回真实 AI 回复	通过（约 4.7 秒）

8.2 真实 API 调用记录

- 调用接口：POST https://api.deepseek.com/v1/chat/completions

- 实测耗时：约 4.7 秒（单次 4708 ms）
- Token 用量：prompt 18 + completion 663 = 681 tokens
- 返回质量：结构清晰、内容详实、可核验

九、AI 协同开发声明与 Prompt 调优策略

9.1 AI 协同贡献说明

本项目采用 DeepSeek 等 AI 辅助编程工具（Vibe Coding）进行开发。AI 主要用于：代码骨架与页面布局生成、基础交互逻辑编写、文档大纲起草。参赛选手全程理解并掌控项目核心架构逻辑，对 AI 生成内容进行了逐项人工审核、调试与修改。关键逻辑（如 DeepSeek API 调用、超时重试、失败降级、密钥管理）均由人工确认。

9.2 Prompt 调优策略

- 模块化拆解：将需求按“项目骨架 / 前端布局 / 工作流可视化 / 聊天逻辑 / 后端接入”拆分，分模块逐步生成。
- 约束先行：明确限定“只写静态结构、不写交互逻辑”等边界，避免一次性生成难以审核的大段代码。
- 迭代修正：对空输入、连续发送等边界情况，通过“发现问题 → 描述问题 → 修复验证”的闭环持续优化。
- 人工介入：配色、文案、动画时长等视觉与体验细节由人工调整，确保作品符合个人审美与赛题要求。

9.3 人工审核与调试记录

通过 Git 细粒度提交历史完整保留开发过程，每次提交对应一个功能模块或一次边界问题修复，官网可据此核查作品原创性与开发过程可复现性。

十、创新点总结

- 多智能体协作 + RAG 的双重抗幻觉机制，切中学术场景对“确定性”的极致要求。
- 工程透明化设计：将智能体运行过程与引用来源可视化，突破传统 AI 对话的黑盒体验。
- 失败自动降级的工程素养，保证作品在评审现场稳定运行。

十一、未来展望

- 接入真实 Embedding 模型与 PDF 解析（含扫描件 OCR），补全「模块一」细粒度文档解析能力。
- 引入 Function Calling / 工具调用，对接 arXiv、PubMed 等外部 API，实现自动检索与追踪。
- 构建局域引用图谱，基于图论算法挖掘“基石论文”，自动生成领域综述。
- 实现 Token 级流式输出与中英双向联动阅读，提升学术级交互体验。

十二、复现指南

1. 安装依赖：pip install -r requirements.txt
2. 配置密钥：复制 backend/.env.example 为 backend/.env，填入 DeepSeek API Key
3. 启动后端：cd backend && uvicorn main:app --reload
4. 打开前端：浏览器打开 frontend/index.html

代码仓库（Public）：https://github.com/J0727/-ai-